

COL380_A4_REPORT

Parallelization Strategy and Performance Analysis:

In Assignment 4, I adopted a multi-layered parallelization approach to exploit distributed-memory, shared-memory, and accelerator resources simultaneously. At the highest level, I use MPI to distribute the work of pairwise block-matrix multiplications across multiple processes. Each MPI rank receives a roughly equal “slice” of the overall set of block-pair tasks, ensuring that no single node becomes a bottleneck. Within each process, I then employ OpenMP to further subdivide the local tasks among all available CPU cores, dynamically scheduling fine-grained units of work to balance load across threads and to overlap CPU preparation (flattening input blocks) with GPU invocation.

I realized that offloading every individual block-multiply to the GPU as a separate kernel launch and memcpy operation was a major performance impediment. To address this, I moved to a “fused”-task kernel that ingests every local rank’s block-pair list in one host-to-device transfer and produces all partial block products in a single, two-dimensional CUDA launch. Within each CUDA block, I load the two $k \times k$ input subblocks into shared memory once, enabling each thread to compute one element of the product using only on-chip memory. This drastically reduces global-memory traffic by approximately a factor of k and eliminates driver and PCIe latency from hundreds of small transfers and launches.

Once the GPU kernel finishes, each MPI rank holds a contiguous buffer of its computed partial blocks. I then perform a host-side accumulation parallelized with OpenMP to sum these partial results into a flat, per-block accumulator array. Finally, an MPI_Allreduce merges these accumulators across ranks in a single collective operation, ensuring that the slowest process governs total compute time. By structuring the algorithm this way, I limit inter-node communication to exactly one global reduction per matrix multiplication stage, and I balance compute and communication phases tightly.

To gauge the effectiveness of these optimizations, I measured wall clock times on our medium and large test cases, including end-to-end I/O. For the medium case (approximately a few dozen sparse blocks per matrix), the unoptimized code took over 20 s, primarily in GPU launch overhead and host device transfers. After moving to the fused-task kernel and eliminating per-block allocations, total time fell to approximately 250 ms, with file-reading and matrix loading accounting for roughly 50 ms, GPU compute about 180 ms, and file writing under 20 ms. For the large case (several hundred blocks), I observed an initial runtime of more than 200 s; with my

full optimization suite, that time dropped to about 13 s, dominated by the GPU compute phase (12 s) and a modest overhead for I/O (1 s).

These results show a speedup of 30x-50x on the medium test and 15x on the large test over the naive implementation. The timings further demonstrate that as the problem scale increases, my communication and load-balancing strategy single collective per multiplication and dynamic OpenMP scheduling keeps overhead sublinear in the number of blocks.